

Corso di Calcolo Parallelo

Laurea Magistrale in Informatica—Università di Bologna

Prova scritta, 10 Giugno 2026

- La prova dura 90 minuti
- Durante la prova non è consentito consultare libri, appunti o altro materiale.
- Non è consentito l'uso di dispositivi elettronici (ad esempio, cellulari, tablet, smart watch/smart glass, ...), né interagire in alcun modo con gli altri ammessi alla prova pena l'esclusione, che potrà avvenire anche dopo il termine della stessa.
- Le risposte devono essere scritte **a penna** su questi fogli, in modo **leggibile**. Le parti illeggibili o scritte a matita saranno ignorate.
- Eventuali altri fogli possono essere utilizzati per la brutta copia ma non verranno valutati.
- Per ritirarsi è sufficiente scrivere "RITIRATO" sulla prima pagina di questo plico.
- I voti saranno pubblicati su AlmaEsami.
- I voti della prova scritta e del progetto restano validi fino alla sessione d'esame di **febbraio 2027** inclusa. Dopo tale data tutti i voti in sospeso saranno persi.

NON SCRIVERE NELLA TABELLA SOTTOSTANTE

D. 1	D. 2	D. 3	D. 4	D. 5
/ 6	/ 6	/ 6	/ 6	/ 6

Domanda 0.

Scrivere nome, cognome e numero di matricola in cima al primo foglio.

Domanda 1.

Discutere le ragioni per le quali, nella pratica, si osserva raramente uno speedup ideale.

Risposta.

Il motivo principale è la presenza di overhead di vario tipo (es., gestione del team di thread in una architettura a memoria condivisa, comunicazioni nelle architetture a memoria distribuita); la legge di Amdahl formalizza il concetto. Abbiamo anche visto, usando l'analisi teorica di work e span, che certi algoritmi non possono avere speedup ideale nemmeno assumendo zero overhead. Un esempio è l'algoritmo per il calcolo della riduzione mediante comunicazioni "ad albero". Tale algoritmo, eseguito con P processori, ha $\text{work} = P$ e $\text{span} = \log P$; lo speedup è quindi $P / \log P = o(P)$ (la notazione " $o(P)$ " indica che il risultato è asintoticamente meno che lineare).

Domanda 2.

Discutere se e come sia possibile parallelizzare il frammento di codice seguente aggiungendo opportuni costrutti OpenMP, senza stravolgere il programma. Solo nel caso in cui il codice possa essere parallelizzato, discutere le eventuali limitazioni della soluzione proposta.

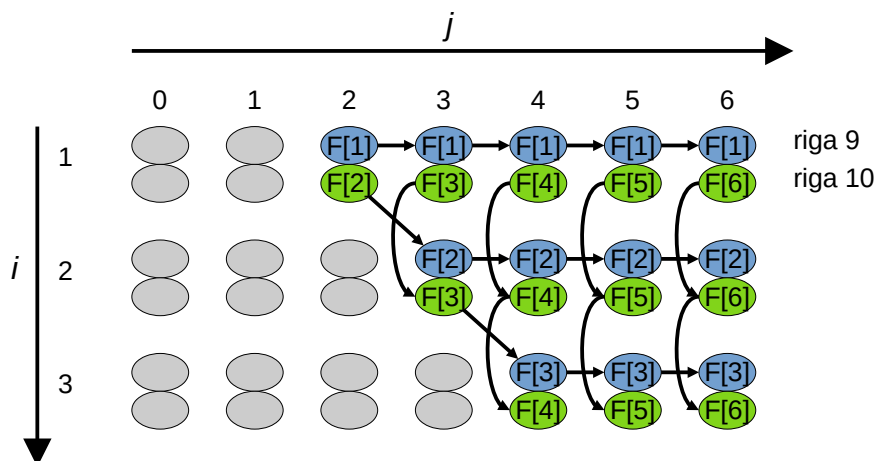
Assumere che $\text{fun}(i, j)$ restituisca un valore che dipende solo dai valori di i e j , e non dal contenuto dell'array $F[]$.

```
1  #define N ...
2  float F[N];
3
4  /* Inizializzazione di F[N] non riportata */
5
6  for (int i=0; i<N; i++) {
7      for (int j=i+1; j<N; j++) {
8          const float f = fun(i, j);
9          F[i] += f;
10         F[j] += f;
11     }
12 }
```

Risposta.

Una premessa: una *race condition* è una *loop-carried dependence*, quindi non è corretto affermare, ad esempio, che entrambi i cicli possono essere parallelizzati perché non ci sono loop-carried dependences. Si può osservare (vedi seguito) che l'aggiunta di una clausola `#pragma omp parallel for` da sola prima di uno dei due cicli non produce un risultato corretto, proprio perché ci sono loop-carried dependences che vanno gestite opportunamente.

Disegniamo ora il diagramma delle dipendenze; le istruzioni importanti sono quelle nelle righe 9 e 10, e per visualizzare meglio cosa succede utilizziamo colori diversi, e indichiamo all'interno degli ovali quali elementi dell'array $F[]$ vengono modificati. Diamo anche dei valori numerici per le variabili indice i e j .



Dal diagramma possiamo osservare come esistano delle dipendenze sia tra iterazioni del ciclo

esterno (quello alla riga 6), perché ci sono delle frecce in verticale che attraversano le iterazioni su i ; inoltre, l'istruzione alla riga 9 introduce delle dipendenze in orizzontale anche tra iterazioni diverse del ciclo interno su j (quello alla riga 7). Di conseguenza, non è possibile parallelizzare banalmente con una clausola `#pragma omp parallel for` né il ciclo alla riga 6 né quello alla riga 7, a meno che non si riescano a gestire in qualche modo le dipendenze.

Dal diagramma precedente si può osservare che, se non ci fosse la riga 9, sarebbe possibile parallelizzare il ciclo interno perché tra gli ovali verdi non ci sono dipendenze in orizzontale. Guardiamo in dettaglio cosa fa la riga 9: essa accumula dei valori su una variabile $F[i]$ che rimane la stessa per tutte le iterazioni del ciclo interno. Questo suggerisce l'uso di una clausola `reduction`: una possibile soluzione che è stata considerata corretta è la seguente (a causa di un cavillo nella specifica di OpenMP¹ non funziona, ma poiché non ne abbiamo parlato a lezione il problema poteva essere ignorato):

```
1  for (int i=0; i<N; i++) {
2  #pragma omp parallel for reduction(+:F[i])
3      for (int j=i+1; j<N; j++) {
4          const float f = fun(i, j);
5          F[i] += f;
6          F[j] += f;
7      }
8  }
```

La soluzione precedente potrebbe essere inefficiente nel caso in cui l'implementazione di OpenMP crei e distrugga il team di thread per ogni iterazione del ciclo esterno. Per evitare questo problema è possibile disaccoppiare il costrutto “parallel” da “for” (attenzione che ciò non è sempre possibile!):

```
1  #pragma omp parallel
2  for (int i=0; i<N; i++) {
3  #pragma omp for reduction(+:F[i])
4      for (int j=i+1; j<N; j++) {
5          const float f = fun(i, j);
6          F[i] += f;
7          F[j] += f;
8      }
9  }
```

Una soluzione che funziona correttamente è la seguente, che risulta inutilmente complicata perché richiede l'introduzione di una nuova variabile (nel nostro caso chiamata Fi) su cui effettuare la riduzione di $F[i]$; il valore di Fi viene poi copiato in $F[i]$ alla termine del ciclo interno.

```
1  for (int i=0; i<N; i++) {
2      float Fi = F[i];
3  #pragma omp parallel for reduction(+:Fi)
4      for (int j=i+1; j<N; j++) {
5          const float f = fun(i, j);
6          Fi += f;
7          F[j] += f;
8      }
9  }
```

1 Secondo la specifica, se la variabile di una clausola *reduction* è un singolo elemento di un array, allora il comportamento è indefinito nel caso in cui all'interno del ciclo si acceda anche ad altri elementi dello stesso array. Non so quale sia la ragione di questa limitazione, che a me appare senza senso.

```

9     F[i] = Fi;
10 }

```

Si noti come in questo caso risulti più complicato disaccoppiare le clausole *parallel* e *for*.

È stata considerata corretta (= punteggio pieno, tranne quando erano presenti altri errori) anche la versione seguente, perché risponde al quesito e produce il risultato corretto, sebbene verosimilmente non risulti efficiente:

```

1  #pragma omp parallel for
2  for (int i=0; i<N; i++) {
3      for (int j=i+1; j<N; j++) {
4          const float f = fun(i, j);
5          #pragma omp atomic
6              F[i] += f;
7          #pragma omp atomic
8              F[j] += f;
9      }
10 }

```

Le *race condition* vengono evitate applicando la clausola *atomic*. Questa versione consente un limitato livello di parallelismo nel caso in cui la valutazione della funzione *fun()* sia computazionalmente onerosa.

Infine riportiamo per completezza un'altra soluzione, che non era praticabile dato che usa la *array reduction* OpenMP, **non** discussa a lezione e quindi **non** facente parte del programma d'esame. Il codice seriale modifica il contenuto dell'array *F[]* accumulando dei valori *fun(i, j)* che, da ipotesi, non dipendono dal contenuto dell'array *F[]*. Di conseguenza, la *array reduction* gestisce correttamente questo scenario, creando degli array locali per ogni thread su cui accumulare i valori *fun(i, j)*, per poi procedere ad una riduzione globale tra gli array locali alla fine della regione parallela.

```

1  #pragma omp parallel for reduction(+:F[:N])
2  for (int i=0; i<N; i++) {
3      for (int j=i+1; j<N; j++) {
4          const float f = fun(i, j);
5          F[i] += f;
6          F[j] += f;
7      }
8  }

```

Dato che il numero di iterazioni del ciclo interno dipende da *i*, la soluzione precedente potrebbe comportare sbilanciamento del carico che può essere risolto specificando la dimensione del chunk e/o usando uno *schedule* di tipo dinamico.

Domanda 3.

Descrivere un programma MPI per calcolare il prodotto $c[N]$ tra una matrice quadrata $A[N][N]$ e un vettore $b[N]$, entrambi di tipo *float*. Ricordiamo che è possibile calcolare il prodotto matrice-vettore con il seguente frammento di codice seriale:

```
1  for (int i=0; i<N; i++) {  
2      c[i] = 0;  
3      for (int j=0; j<N; j++)  
4          c[i] += A[i][j] * b[j];  
5  }
```

Si assuma che:

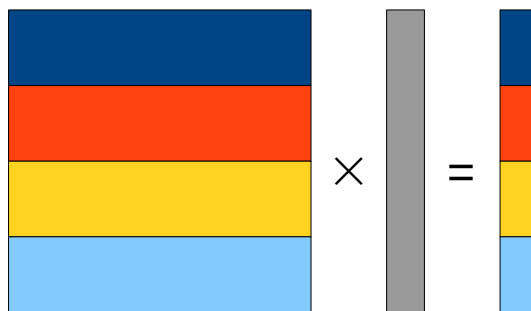
- La dimensione N della matrice A e del vettore b siano inizialmente noti a tutti i processi;
- N sia un multiplo del numero di processi MPI;
- Il contenuto di $A[i][j]$ e $b[j]$ siano inizialmente noti solo al processo 0;
- Al termine della computazione, il risultato $c[i]$ debba essere noto al processo 0.

Risposta.

Un programma funzionante che risolve questo problema è disponibile nei “*Parallel Etudes*” con il nome `mpi-gemv.c`.

L’idea è quella illustrata a lezione nelle slide sui pattern per la programmazione parallela: distribuire blocchi di A composti da N/P righe di N elementi ciascuna, mediante l’operazione `MPI_Scatter()`, e assegnare a ciascun processo una copia intera dell’array b tramite una operazione `MPI_Bcast()`. Ciascun processo calcolerà una porzione del risultato c che potrà poi essere concatenato.

Si presti attenzione che per distribuire la matrice A occorre inviare a ciascun processo $(N/P)*N$ elementi di tipo *float*.



Domanda 4.

1. Illustrare brevemente i principali tipi di memoria presenti in una GPU CUDA-
2. Perché c'è una gerarchia così complessa? Non sarebbe stata sufficiente solo la memoria globale? Discutere.

Risposta.

La ragione per cui esiste una gerarchia di memoria complessa nelle GPU è il cosiddetto “memory wall” di cui abbiamo parlato nella lezione dedicata alle architetture parallele.

Domanda 5.

La funzione seguente, vista a lezione, calcola la somma del contenuto di un array $v[]$ di lunghezza n senza usare la clausola `reduction()`.

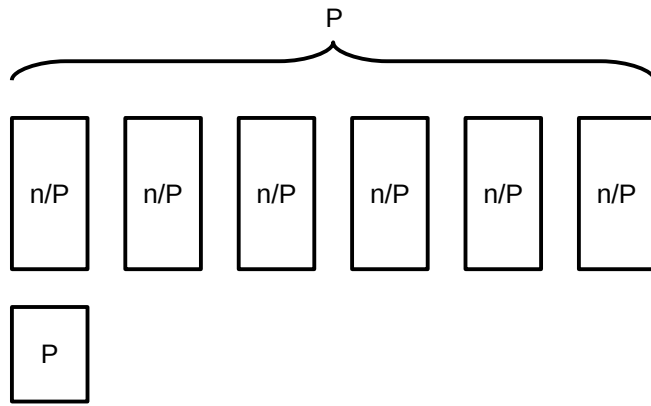
```
1  long arraysum(const int *v, int n)
2  {
3      const int P = omp_get_max_threads();
4      long local_sum[P];
5      long sum = 0;
6      #pragma omp parallel
7      {
8          const int my_id = omp_get_thread_num();
9          const int my_start = (n * my_id) / P;
10         const int my_end = (n * (my_id+1)) / P;
11         local_sum[my_id] = 0;
12         for (int i=my_start; i<my_end; i++)
13             local_sum[my_id] += v[i];
14     }
15     for (int k=0; k<P; k++)
16         sum += local_sum[k];
17     return sum;
18 }
```

Assumere che il *pragma* alla riga 6 crei P thread OpenMP, dove P è il valore calcolato alla riga 3.

1. Determinare *work*, *span* e *speedup* della funzione `arraysum()` in funzione della lunghezza n dell'array e del numero P di unità di esecuzione; assumere che sia n che P possano variare arbitrariamente, quindi le espressioni calcolate potranno dipendere sia da n che da P . Motivare la risposta.
2. Discutere lo *speedup* nel caso (ipotetico) in cui $P = n$. Cosa succede se n tende all'infinito?
3. Discutere lo *speedup* nel caso (più realistico) in cui P sia una costante "piccola". Cosa succede in questo caso se n tende all'infinito? (*Suggerimento: per calcolare il limite per x che tende a infinito di un rapporto del tipo $\frac{ax+b}{cx+d}$ si può raccogliere l'incognita ottenendo $\frac{x(a+b/x)}{x(c+d/x)}$ e quindi semplificare x*).

Risposta.

Ciascuna istanza del blocco parallelo alle righe 7--14 richiede $\text{work} = \text{span} = n/P$ e viene effettuato in parallelo da P processi. Il ciclo alla riga 15 viene eseguito da un singolo processo, e richiede $\text{work} = \text{span} = P$. Combinando queste osservazioni possiamo effettuare l'analisi grafica mostrata in figura.



Relativamente al punto 1, abbiamo:

$$\text{Work} = (n/P \times P) + P = n+P$$

$$\text{Span} = \frac{n}{P} + P$$

$$\text{Speedup} = \frac{\text{Work}}{\text{Span}} = \frac{n+P}{\frac{n}{P} + P}$$

Relativamente al punto 2, se $P = n$ possiamo riscrivere l'espressione dello speedup in funzione solo di P o di n tramite sostituzione:

$$\text{Speedup} = \frac{2n}{1+n}$$

Se n tende all'infinito, possiamo calcolare il limite dello speedup raccogliendo la variabile n al denominatore e denominatore sfruttando il suggerimento dato nel punto 3:

$$\lim_{n \rightarrow \infty} \frac{2n}{1+n} = \lim_{n \rightarrow \infty} \frac{2n}{n(\frac{1}{n}+1)} = \lim_{n \rightarrow \infty} \frac{2}{\frac{1}{n}+1} = O(1)$$

Lo speedup con il massimo numero possibile di processori è $O(1)$, quindi l'algoritmo non è scalabile in questo scenario (lo speedup ottimale dovrebbe essere n oppure P dato che assumiamo che siano uguali). Il collo di bottiglia è il ciclo della riga 15 il cui corpo viene eseguito serialmente $n = P$ volte.

Relativamente al punto 3, se P è $o(n)$, possiamo calcolare il limite dello speedup trattando P come una costante:

$$\lim_{n \rightarrow \infty} \frac{n+P}{\frac{n}{P} + P} = \lim_{n \rightarrow \infty} \frac{n(1+\frac{P}{n})}{n(\frac{1}{P} + \frac{P}{n})} = \lim_{n \rightarrow \infty} \frac{1+\frac{P}{n}}{\frac{1}{P} + \frac{P}{n}} = \frac{1}{\frac{1}{P}} = P$$

In questo caso osserviamo che lo speedup asintotico è pari a P , che è il valore ottimo. Di conseguenza, la funzione risulta scalabile nel caso in cui il numero di processori P sia una costante molto minore di n , che è quello che accade nella pratica.